

MODULE 3 · TOPIC 1

Overview of VLSI STA

Introduction to timing analysis · Combinational circuit timing: races and hazards · Sequential circuit timing: setup and hold · Maximum frequency of operation · Timing constraints for synthesis

- Theory 1 · What timing analysis is, and what a truth table cannot tell you
- Theory 2 · Races and hazards — the new material, and most of the session
- Theory 3 · Setup, hold, maximum frequency, and real violations
- Theory 4 · Timing constraints for synthesis, and what synthesis does back
- Practical · Labs A-G · 12 hours · 58 exercises · Vivado on Zynq-7000



Terminal Outcomes

Module 3 terminal outcomes

NOS NIE/ELE/N0103 · Static Timing Analysis of VLSI Circuits · 25 h theory + 35 h practical

1

Understand static timing analysis

what a timing path is, how arrival, required and slack are computed, and how to read any timing report

2

Understand ECO fixes and timing closure

how to diagnose a violation, choose a fix, and drive a design to a state where every check passes at every corner

Where this subtopic sits

Subtopic 1 · Overview of VLSI STA
Subtopic 2 · Timing performance
Subtopic 3 · STA using EDA tools

builds outcome 1, and the vocabulary outcome 2 needs
pipelining, retiming, skew, block and chip level
post-synthesis and post-route, ECO flow, sign-off

This deck is subtopic 1. It ends where the ECO flow begins.

This subtopic is the foundation for both of them

Outcome 1 is built here and completed in subtopic 3. Outcome 2 needs the vocabulary and the diagnostic habits this subtopic establishes — you cannot write an ECO for a violation you cannot classify.



Key Learning Outcomes

Key learning outcomes for this subtopic

THEORY

- Describe the key components of STA: timing paths, constraints, reports
- Analyse timing issues in combinational circuits, including races and hazards
- Discuss how timing constraints influence synthesis and optimisation
- Evaluate maximum frequency of operation

PRACTICAL

- Conduct STA using industry-standard tools to analyse timing paths
- Identify and resolve setup and hold violations through exercises
- Implement timing constraints in EDA tools and analyse their impact during synthesis
- Evaluate the effect of clock skew

Every one of them is assessed by something you run

races and hazards -> make analyse, make glitch, make capture
constraints and synthesis -> make synth, make setup
setup and hold -> make hold, and the Fmax sweep
industry-standard tools -> vivado/zynq_sta.tcl on a Zynq-7000

Every outcome on that slide is assessed by something you run, not by something you recite.



Every Syllabus Phrase, and Where It Is Covered

Every syllabus phrase, and where it is covered

syllabus phrase	section	slides
Introduction to timing analysis	Theory 1	5-12
Combinational circuit timing - races and hazards	Theory 2	13-32
Sequential circuit timing - setup and hold timing	Theory 3	33-44
Maximum frequency of operation	Theory 3	45-50
Practical examples of setup and hold violations and their solution	Theory 3 + Labs F	51-58
Timing constraints for synthesis	Theory 4	59-66
Circuit synthesis and timing analysis	Theory 4	67-72

On the overlap with Module 2 Topic 6

Setup, hold and constraints were taught there from the RTL designer's point of view. Here they are revisited at gate level and treated as the second half of a bigger question. The genuinely new material in this subtopic is races and hazards - and it is given the most room.

The genuinely new material in this subtopic is races and hazards, so it is given the most room.



How This Session Runs

How this session runs

THEORY 1	What timing analysis is why a truth table is not enough
THEORY 2	Races and hazards the new material - most of the session
THEORY 3	Setup, hold, Fmax the sequential half, at gate level
THEORY 4	Constraints and synthesis what the tool does with it all
PRACTICAL	Labs A-G 12 hours, 58 exercises, all measured
One sentence to open with A circuit can compute exactly the right answer and still be wrong, because it passed through a wrong answer on the way - and no truth table and no static analysis will tell you.	

A note on overlap with Module 2 Topic 6

Setup, hold and constraints were taught there from the RTL designer's point of view. Here they return at gate level, as one half of a larger question — because a circuit can pass every timing check and still misbehave.

THEORY 1

What Timing Analysis Is

A truth table tells you what a circuit computes. It says nothing at all about when.

- Why a correct circuit can still be wrong
- The three questions timing asks
- Static analysis and dynamic simulation — two different questions
- The vocabulary the rest of the session assumes



1.1 · WHY

A Circuit Can Be Correct and Still Be Wrong

A truth table is a promise about **settled values**. It says that once everything has stopped moving, the output will be this. It makes no promise whatsoever about **the journey** — and real gates take real time to make that journey.

What the truth table guarantees

Given inputs A, B, C, the output will eventually be F.

Every row, every time.

This is what your RTL simulation checks.

What it does not guarantee

That F goes there directly.

That F does not visit the wrong value first.

That F arrives before the next clock edge.

And what neither one checks

Whether the value that mattered was sampled at the right instant.

A design can be functionally perfect and electrically unusable.

Two failures that a functional simulation will never show you

1. The output passes through a wrong value on its way to the right one — a HAZARD.
2. The output arrives after the flip-flop needed it — a SETUP VIOLATION.

Both are invisible in a zero-delay simulation, because a zero-delay simulation has no delays to get wrong.



Timing Analysis Asks Three Things

The question	The failure if the answer is no	Answered by
Did the data arrive before the capturing edge needed it?	setup violation — the chip runs, but slower	static timing analysis
Did the data stay put long enough after that edge?	hold violation — the chip does not work at all	static timing analysis
Did the output pass through a wrong value on the way?	a hazard — a glitch, harmless or fatal depending on where it goes	simulation with delays

The first two are about ONE path. The third is about TWO.
Setup and hold ask how long a path is. A hazard asks what happens when two paths of different length reconverge — so no measurement of any single path can detect one.
That is why this topic needs two different tools on the same circuit, and why a flow with only one of them has a blind spot.

Module 2 Topic 6 answered the first two thoroughly. This topic starts with the third.



Two Tools, One Circuit, Two Different Questions

Two tools, one circuit, two different questions

STATIC TIMING ANALYSIS

sta/sta.py, PrimeTime, Vivado

- How long is the longest path?
- How long is the shortest path?
- Does every path fit in a clock cycle?

sees setup and hold violations

is BLIND to every hazard

GATE-LEVEL SIMULATION

iverilog with delays

- What does the output do over time?
- Does it pass through a wrong value?
- How many times does it change?

sees every glitch you stimulate

misses any path you never exercise

Neither tool is a superset of the other

Run STA on hz_static1.v and every path meets timing, because every path does meet timing. The glitch is still there. Run a zero-delay RTL simulation and the glitch is not there either, because zero-delay simulation has no delays to race.

You need the delay-annotated simulation for hazards, and STA for setup and hold. That is why this lab uses both on the same circuit.

Run STA on the hazard circuit and every path meets timing, because every path does meet timing. The glitch is still there.



The Words the Rest of the Session Assumes

Term	Meaning
combinational logic	output depends only on the present inputs
sequential logic	output depends on the inputs and on stored state
propagation delay	how long a gate takes to react to its input
timing path	a route from a start point to an end point, with a delay
arrival time	when the data actually reaches a pin
required time	when it had to be there for the check to pass
slack	required minus arrival; negative means it does not fit
setup time	how long D must be stable BEFORE the clock edge
hold time	how long D must stay stable AFTER the clock edge
clock skew	the same edge reaching two registers at different instants
glitch	a momentary wrong value on an output
hazard	a circuit's POTENTIAL to glitch, given unlucky delays
race	two signals changing where the order decides the outcome



1.5 · A DISTINCTION

Hazard and Glitch Are Not the Same Word

HAZARD

A property of the CIRCUIT.

It means: there exists an assignment of gate delays for which this output glitches.

You can find it by inspecting the logic, with no simulation at all — which is what tools/hazard.py does.

It is either there or it is not.

GLITCH

An EVENT in one simulation.

It means: on this run, with these delays, the output actually moved when it should not have.

You can only see it by simulating with delays, and only on the transitions you stimulated.

It may or may not show up.

Why the distinction earns its keep

A circuit with a hazard may run for years without glitching, because the delays happened to fall the right way — until a new process corner, a new temperature, or a re-synthesis changes them. You remove hazards, not glitches.

THEORY 2

Combinational Circuit Timing: Races and Hazards

The part of this subtopic that Module 2 never covered, and the part that static timing analysis is structurally unable to see.

- What a hazard is, and why it is a property of the circuit
- Static-1, static-0, dynamic — and the function hazard you cannot fix
- Finding them: the adjacency rule, on a K-map and in code
- Removing them: the consensus term, and where it stops working
- Races, and why synchronous design mostly makes them go away



2.1 · THE IDEA

A Wrong Value On the Way To the Right One

A hazard is a wrong value on the way to the right one

The truth table says the output should not move. For a few hundred picoseconds, it does.

what the truth table promises



F stays 1

what the gates actually do



the glitch

Why it happens: two paths, one output

One input feeds the output by two different routes with different delays. When that input changes, one route has already reacted and the other has not. The output is briefly computing from two inconsistent versions of the same signal.

This is not a slow path, and static timing analysis cannot see it

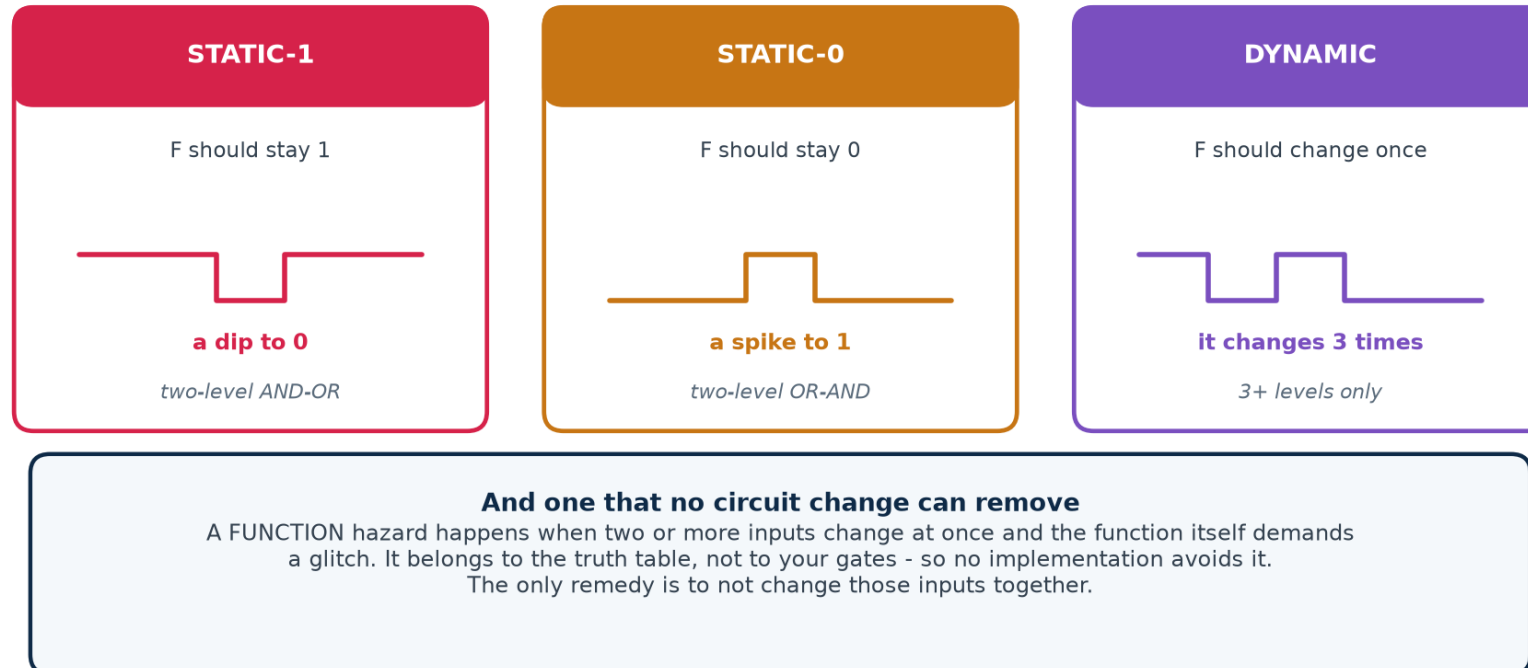
STA measures how long each path is, and every path here meets timing. A hazard is about the DIFFERENCE between two paths, so only a simulation with delays will show it.

Two paths from one input, different delays, one output. That is the entire mechanism.



Three Kinds of Hazard, and One You Cannot Remove

Three kinds of hazard



Logic hazards come from your implementation and can be removed. A function hazard belongs to the truth table itself.



2.3 · FUNCTION HAZARDS

The One No Implementation Can Fix

Everything else in this section assumes **one input changes at a time**. Let two change at once and a new problem appears that no amount of redundant logic will touch.

two inputs changing at once

$F = A B' + B C$ with $A = 1, C = 0$

```
start  A=1 B=1 C=0  -> F = 0
end    A=1 B=0 C=1  -> F = 1
```

Two inputs changed. In between, the circuit necessarily passes through either $A=1 B=1 C=1$ ($F = 1$) or $A=1 B=0 C=0$ ($F = 1$), depending which input the wiring happens to deliver first.

Both intermediate states give $F = 1$. The output is SUPPOSED to end at 1, so here it is harmless - but change the example and it need not be.

Why it cannot be fixed

The glitch is in the FUNCTION, not the gates. Every correct implementation must pass through one of those intermediate states, because the inputs really do go through them.

What to do instead

Do not change those inputs simultaneously — which, in a synchronous design, is exactly what a common clock guarantees.

This is one more reason the clocked discipline is worth its cost.

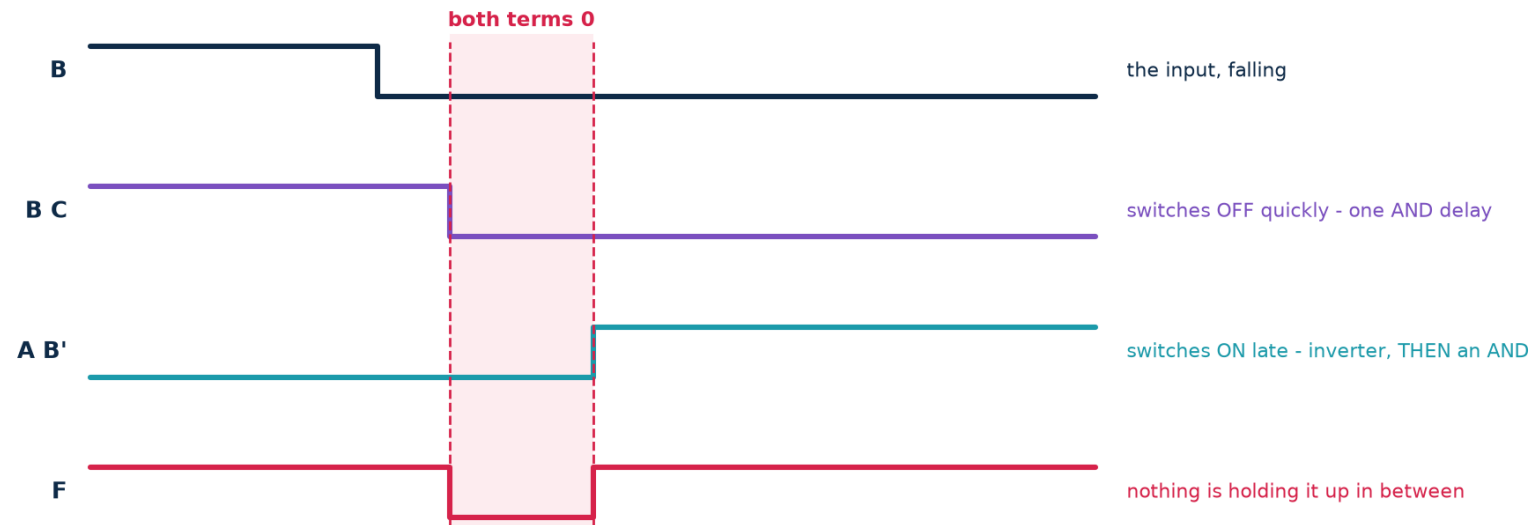
From here on: one input at a time, and every hazard we discuss is a LOGIC hazard.



The Race, In Time

The race, in time: $F = A B' + B C$ with $A = 1, C = 1$

B falls. Watch the two product terms hand over - and miss.



The rule this gives you

Take two input patterns one variable apart that both give $F = 1$. If NO single product term covers both of them, then one term must switch off while another switches on - and whether the output dips is decided by delays you do not control.

That is a static-1 logic hazard, and tools/hazard.py finds them by exactly this test.

B C switches off as soon as B falls. A B' cannot switch on until B has been through the inverter. In between, nothing is holding F up.



2.5 · THE RULE

How To Find One Without Simulating Anything

The adjacency rule, in one sentence

Take two input patterns that differ in exactly ONE variable and both give $F = 1$. If no single product term covers both of them, the circuit has a static-1 logic hazard on that transition.

$F = A B' + B C$ $A=1, C=1$, and B changes

$B=1$: $F = 1$, held up by $B C$

$B=0$: $F = 1$, held up by $A B'$

Does one term cover BOTH?

$A B'$ needs $B=0$ -> covers only the second

$B C$ needs $B=1$ -> covers only the first

No. So the output is handed from one term to the other, and whether it dips depends on which gate wins.

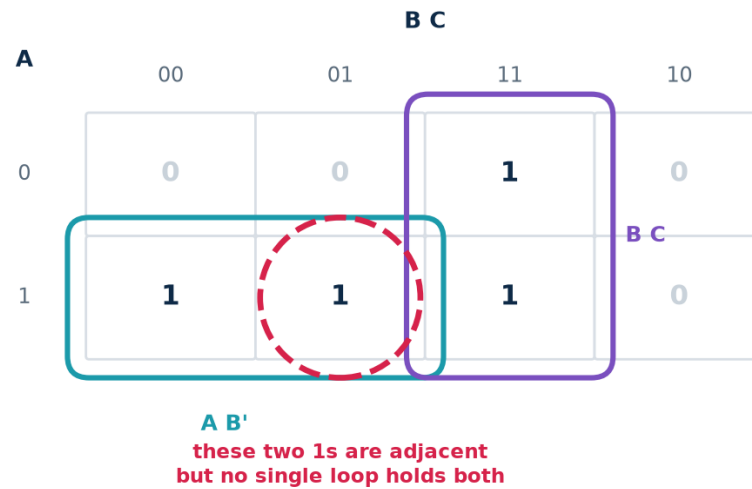
Why this is the right test

If some term is 1 for both patterns, that term does not change at all during the transition — so it holds the output up throughout and no delay assignment can produce a dip. If no term does, then one must switch off while another switches on, and the order is decided by delays you do not control.



Adjacent 1s In Different Loops

The same thing on a K-map: adjacent 1s in different loops



The fix is a third loop that bridges the gap

Draw a loop around those two cells: $A = 1, C = 1, B$ either way. That is the term $A C$.

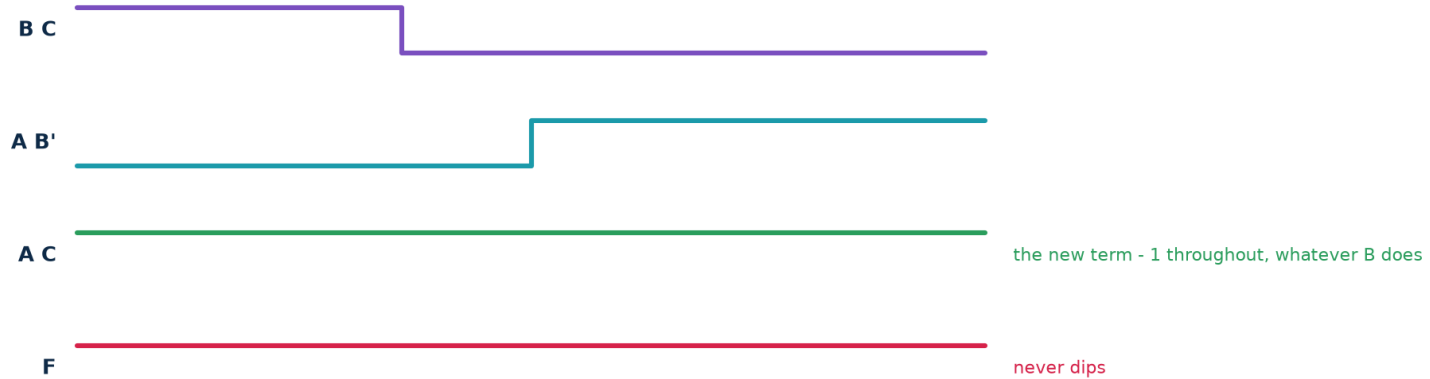
It covers no new 1s, so it changes nothing in the truth table. It is REDUNDANT - and that is exactly why a minimiser deletes it and why you have to ask for it deliberately.

Two 1-cells side by side with no single loop around both. That is the same test, drawn.



The Consensus Term

The consensus term: one extra gate, no logic change



why it works

With $A = 1$ and $C = 1$ the term $A C$ is 1 whatever B is doing. It holds F up right through the transition, so there is no handover to lose.

what it costs

One AND gate and one more input on the OR - and a term a logic minimiser will delete the moment you stop protecting it.

Measured in the lab

hz_static1: 1 glitch over 24 transitions. hz_static1_fix: 0.

Both truth tables 10111000 - the function did not change.

A term that covers no new 1s, changes nothing in the truth table, and holds the output up through the handover.



Redundant Is Not a Criticism — It Is the Requirement

The consensus term is **logically redundant by construction**. Delete it and the truth table is identical. That is not a flaw in the fix — it is what makes the fix safe, and it is also what makes it fragile.

Why redundancy is required

A term that changed the function would not be a fix, it would be a bug.
The only way to hold the output up without altering what the circuit computes is to add cover that is already covered.

Why it will not survive

Removing redundancy is precisely what a logic optimiser exists to do.

Write the consensus term in RTL and synthesis will delete it — measured in Theory 4 of this session.

How to find the term, every time

Take the two input patterns. Write down every variable they agree on, at the value they agree on. Drop the one that changed. That product is the term to add — and tools/hazard.py prescribes it automatically.



Doing It In Code

```
$ python3 tools/hazard.py "A B' + B C"

1 static-1 logic hazard(s) found:

  B changes 1 -> 0   with  A=1, C=1
    before the change, the output is held up by : B C
    after  the change, the output is held up by : A B'
    no single term covers both, so the handover is a race.
    ADD the redundant term:  A C

hazard-free cover:
  F = A B' + B C + A C

checking the proposed cover:
  remaining static-1 hazards : 0
  truth table unchanged      : yes
```

It checks its own answer, and that matters

A tool that proposes a fix without verifying it is a tool that will eventually propose a wrong one. This one re-runs the analysis on the patched cover and re-derives the truth table before it prints.



How We Know the Rule Is Actually Right

The adjacency rule is a claim about **logic**. Whether an output glitches is a fact about **time**. Those are different things, so the analyser's self-test checks one against the other.

```
$ python3 tools/hazard.py --selftest

D. the combinatorial rule agrees with a delay simulation
  (random functions; the rule and the timeline must never differ)
static-1 transitions cross-checked      PASS (got True, want True)
disagreements between rule and simulation PASS (got 0, want 0)

E. the suggested fix always works, and never changes the function
fixes that left a hazard behind          PASS (got 0, want 0)
fixes that altered the truth table        PASS (got 0, want 0)

SELF-TEST PASSED
```

What that test actually does

It generates hundreds of random covers. For every single-variable transition it asks the covering rule, and separately builds a timeline with randomised per-gate delays to see whether a dip is possible. Over several thousand transitions the two answers never differed.

That is the difference between believing a textbook rule and having checked it.



Finding Glitches By Counting, Not By Squinting

Finding glitches by counting, not by squinting

A waveform viewer does not scale and does not go in a regression. Counting does.

1

Settle at the starting input pattern, record F

2

Arm a counter on every change of F

3

Change ONE input, wait, record F again

4

Expected changes: 0 if F ended where it started, else 1

5

More than expected = a glitch. Name it and count it.

It also records the settled truth table

So a "fix" that quietly changes the function cannot pass as a fix. In the lab every design in a family prints the same signature - 10111000 before the fix and after it.

24 transitions examined per design. Three inputs, eight states, three single-variable moves from each.

A waveform viewer does not scale, does not go in a regression, and does not tell you when you have fixed something.



Six Designs, One Detector, and a Surprise

Six designs, one detector, and a surprise in row five

design	cover	truth table	detector
hz_static1	$A B' + B C$	10111000	1 glitch (static)
hz_static1_fix	$+ A C$	10111000	CLEAN
hz_none	$A B + A C$	11100000	CLEAN
hz_dynamic	$(A B' + B C) \text{ XOR } B$	01110100	5 glitches
hz_dynamic_fix	$+ A C \text{ inside}$	01110100	4 glitches
hz_flat_fix	$A B' + B C' + A C'$	01110100	CLEAN

Row five is the one to stop on

Adding the consensus term removed the DYNAMIC hazard and left four static ones. Those have a different cause: with $A=0, C=1$ the inner expression collapses to a DELAYED COPY of B, so the circuit computes $B \text{ XOR } B\text{-delayed}$ and spikes on every edge.

That is reconvergent fanout. No redundant product term repairs it, because the cover is not what is wrong - the structure is. Row six flattens it and re-covers: clean.

"Add the consensus term" cures a two-level logic hazard, and only that.

Read row five before the next slide.



Where "Add the Consensus Term" Stops Working

hz_dynamic feeds the hazardous expression into an XOR where B reconverges. The detector reports five glitching transitions. Adding the consensus term removes **one** of them.

design	glitching transitions	what changed
hz_dynamic	5 (4 static, 1 dynamic)	the starting point
hz_dynamic_fix	4 (4 static)	consensus term added inside
hz_flat_fix	0	flattened to two levels, then re-covered

Why the four survivors do not respond to a redundant term

With A=0, C=1 the inner expression collapses to a delayed copy of B. The circuit is then computing B XOR B-delayed, which spikes on every edge no matter what.

That is reconvergent fanout with unequal path delays. The cover is not what is wrong — the STRUCTURE is, and only restructuring fixes it.

The consensus term cures a two-level logic hazard, and only that. Know the limit of your tool.



One Glitchy Signal, Three Consumers

One glitchy signal, three consumers - the measurement

how the signal was used	what happened	why
as DATA into a normally-clocked flop	correct	sampled long after it settled
as a CLOCK	4 spurious edges	every glitch is an edge
as an ASYNCHRONOUS RESET	flag cleared	no clock referees it

The glitch was placed 80 ns before any clock edge

That is the friendliest possible case for "synchronous design tolerates glitches". It is true for the first row and false for the other two - and the signal is the same signal.

So the rule is not "glitches do not matter"

It is: a glitch is harmless ONLY where a clock edge samples it after it has settled.

Anywhere edge-sensitive - a derived clock, a latch enable, an asynchronous reset or set, a handshake into another clock domain - hazards are still yours to remove.

The glitch was placed 80 ns before any clock edge — the friendliest possible case for the usual reassurance.



The Rule, Stated Properly

Not "glitches do not matter in synchronous design"

A glitch is harmless ONLY where a clock edge samples the signal after it has settled.

Safe: level-sampled by a clock

Data into a normally-clocked flip-flop.

Combinational logic between two registers in the same domain.

Here the clock is the referee, and the glitch has a whole cycle to go away.

Unsafe: anything edge-sensitive

Logic driving a CLOCK — every glitch is a clock edge.

An ASYNCHRONOUS reset or set — no clock referees it.

A LATCH enable — the latch is transparent while it is high.

A handshake crossing into another clock domain.

And one cost that applies everywhere

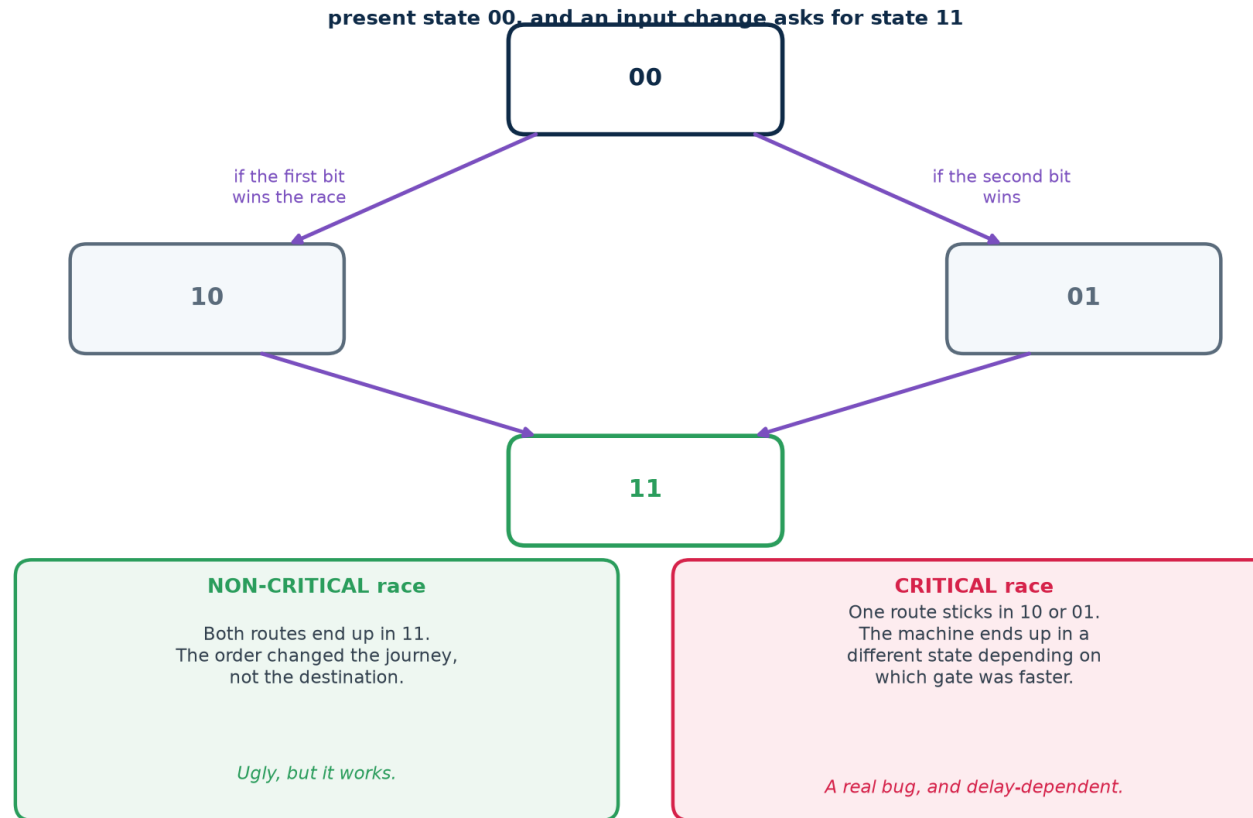
Every glitch charges and discharges capacitance. On a wide bus that switches spuriously, glitch power is real and measurable — a reason to care about hazards even where they are functionally harmless.



When Two State Variables Change At Once

A race: two state variables change, and the order is not fixed

Hazards are about one output. Races are about the state a machine lands in.



A hazard is about one output taking a wrong value. A race is about a machine landing in the wrong STATE.



Critical, Non-Critical, and the Essential Hazard

Name	What happens	Is it a bug?
Non-critical race	the state variables change in either order and the machine still lands in the intended state	no — untidy, but correct
Critical race	the final state depends on which variable changed first	yes — and it is delay-dependent
Essential hazard	an input change races its own inverted copy through the feedback path, and the machine sees the input twice	yes — cured by delay in the feedback, not by logic

Why these are asynchronous problems

All three need state to change in response to an INPUT change. Put a clock in charge and state changes only on an edge, by which time everything has settled — which is why an entire class of problem disappears the moment you adopt synchronous discipline.



What Synchronous Design Buys You, and What It Does Not

Why synchronous design makes races and hazards go away

Asynchronous circuit

state changes whenever an input changes

races and essential hazards are REAL and must be designed out

Synchronous circuit

state changes only on a clock edge

everything settles between edges; the clock is the referee

Which is why this topic spends its time on setup and hold

In a synchronous design the question is never "did it glitch?" - it is "had it stopped glitching by the time the clock edge arrived?". That question is setup and hold, and it is what STA answers.

The exceptions are the places with no clock to referee them: logic that drives a clock, a latch enable, or an asynchronous reset - and any genuinely asynchronous interface. There, hazards are still yours to remove.

It buys you almost everything in this section — except at the boundaries, where there is no clock to referee.



THEORY 2 · CHECKPOINT

Ten Questions Before We Move On

#	Question	The answer in one line
1	Hazard or glitch — which is a property of the circuit?	the hazard; the glitch is one event in one run
2	Which hazard can a two-level AND-OR circuit have?	static-1 only
3	How many levels does a dynamic hazard need?	three or more
4	What is a function hazard?	two inputs change at once and the function itself demands a glitch
5	State the adjacency rule.	two 1-cells one variable apart, covered by no single term
6	How do you build the fixing term?	the literals both patterns agree on, minus the one that changed
7	Why will synthesis delete it?	it is redundant, by design
8	When does the consensus term NOT help?	reconvergent fanout — a structural problem, not a cover one
9	Where is a glitch genuinely dangerous?	clocks, async resets, latch enables, domain crossings
10	Why can STA not find any of this?	it measures one path; a hazard needs two

If question 8 or 10 is fuzzy, say so now — Theory 3 assumes both.

Sequential Circuit Timing

Once a clock is in charge, the question stops being "did it glitch?" and becomes "had it finished glitching by the time the edge arrived?".

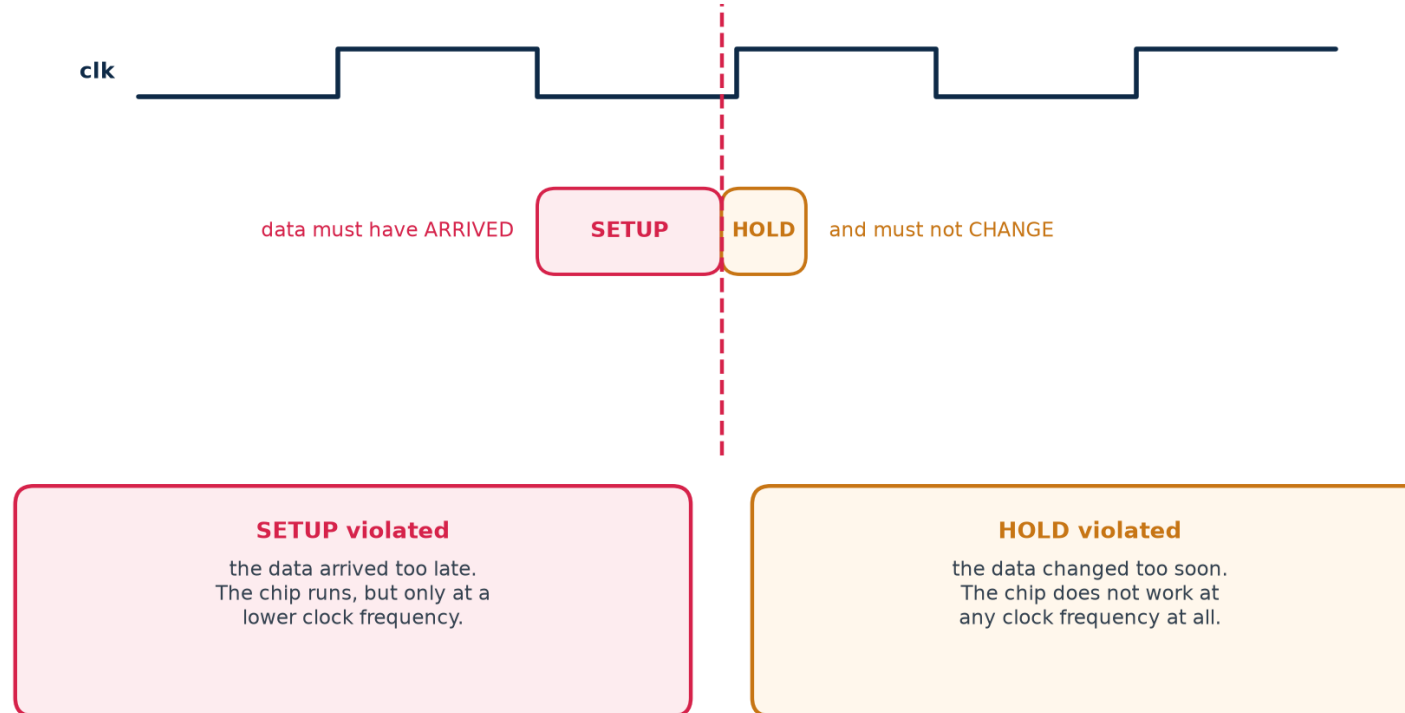
- ❑ The sampling window: setup and hold
- ❑ Maximum frequency of operation, derived rather than quoted
- ❑ A real setup violation and the fix that clears it
- ❑ A real hold violation, and why the clock frequency cannot help



A Flip-Flop Samples Over a Window, Not At an Instant

Sequential timing: the window a flip-flop needs

A flip-flop does not sample AT the edge. It samples over a window around it.



Everything in this section comes from that one picture

Break the first half and the flop captures the previous value — a setup violation. Break the second and it captures the NEXT value, a whole cycle early — a hold violation.



What Consumes the Clock Period

Term	What it is	Who sets it
clock-to-Q	delay from the launching edge until Q is valid	the library cell
logic delay	every gate and wire between the two flops	your design, and synthesis
setup time	how early the capturing flop needs the data	the library cell
clock skew	the difference in clock arrival between the two flops	the clock tree, after layout
uncertainty	jitter, plus skew that is not modelled yet	you, in the constraint file

```
setup:  t_cq + t_logic  <=  T + t_skew - t_setup - t_unc
hold:   t_cq + t_logic  >=  t_skew + t_hold + t_unc

# the clock period T is in the first line and NOT in the second.
# everything surprising about hold follows from that one asymmetry.
```

Note the sign on skew

A capture clock that arrives LATE gives the data more time, so it helps setup — and it eats hold margin by exactly the same amount. Every clock-tree decision trades one against the other.



Maximum Frequency of Operation, Derived

Maximum frequency of operation, derived

The data must arrive before the next edge needs it

$$t_{cq} + t_{logic} \leq T + t_{skew} - t_{setup} - t_{unc}$$

Rearranged, that is the smallest period the design allows

$$T \geq t_{cq} + t_{logic} + t_{setup} + t_{unc} - t_{skew}$$

And the fastest clock is one over that

$$F_{max} = 1 / (t_{cq} + t_{logic} + t_{setup} + t_{unc} - t_{skew})$$

t_logic is the LONGEST path, and only the longest one

Every other path in the design could be twice as fast and Fmax would not move. That is why timing work is always about one path at a time - and why fixing it usually reveals the next one.

Note the sign on skew: a capture clock that arrives LATE gives the data more time, so it raises Fmax. It also eats your hold margin by exactly the same amount.

Not quoted from a datasheet — rearranged from the setup inequality you just saw.



Worked, With Real Numbers

A path in the teaching library:

clock-to-Q (DFF)	0.145 ns
logic: one XOR2 driving a flop D pin	0.117 ns
setup time (DFF)	0.090 ns
uncertainty	0.000 ns
skew	0.000 ns

minimum period	0.352 ns

$F_{\max} = 1 / 0.352 \text{ ns} = 2841 \text{ MHz}$

Add 0.15 ns of uncertainty and the same path gives 1992 MHz.
The circuit did not change. Your honesty about the clock did.

Which is why an Fmax figure without its assumptions is marketing

Ask what corner, what uncertainty, and whether it is post-synthesis or post-route. The same netlist will happily produce two numbers a factor of two apart under different answers to those three questions.



Fmax Is Set By One Stage — The Measurement

Fmax is set by ONE stage - the measurement

pipe_unbal - one stage carries most of the work



the clock has to fit the WIDEST box, not the average one

design	what changed	longest path	Fmax
pipe_unbal	one heavy stage	2.742 ns	364.7 MHz
pipe_bal	heavy stage split in two	2.113 ns	473.2 MHz

1.30x the clock, from adding one register

Same arithmetic, same result, one more cycle of latency. Nothing was made faster - the work was just distributed so that no single stage sets a slow speed limit for all of them.

364.7 MHz to 473.2 MHz, by moving work rather than by making anything faster.



What Pipelining Costs, and What It Does Not

What you gain

A shorter longest path, so a faster clock.

Throughput is unchanged once the pipe is full — one result per cycle, still.

Measured here: 1.30x.

What you pay

One more cycle of latency.

One more bank of registers — area and power.

And every control signal travelling beside the data must be delayed to match, or the design meets timing and computes the wrong answer.

The classic pipelining bug

You cut the datapath and forget to delay a valid flag, an operand or an address alongside it. Timing improves; correctness quietly does not. Every optimisation needs a functional re-check — which is why the Module 2 lab ends with make verify.



A Real Setup Violation, and the Fix That Clears It

A real setup violation, and the fix that clears it

Same constraint file, same 400 MHz target, two versions of the same design.

design	target	worst slack	verdict
pipe_unbal	400 MHz	-0.322 ns	VIOLATED
pipe_bal	400 MHz	+0.307 ns	MET

Read the two slack numbers as a fraction of the period

The period is 2.5 ns. -0.322 ns is 13% over budget - too much for a synthesis option to find, and about right for one pipeline cut. That arithmetic tells you which fix to reach for before you try any of them.

The fix was not a faster tool setting. It was one more register.

Splitting the heavy stage moved the whole design from failing to passing, at the same frequency.

Same constraint file, same target, two versions of the same arithmetic.



Sizing the Problem Before Choosing the Fix

Slack as a fraction of the period	What that usually means	Where to start
under about 2%	the tool nearly got there	check the constraint, then raise synthesis effort
2% to 10%	a genuinely long path	restructure the logic, or retime
10% to 40%	too much work in one cycle	pipeline it
over about 40%	the target is wrong for this technology	renegotiate the frequency, or change architecture

The measured case

−0.322 ns on a 2.5 ns period is 13% over budget. That is out of reach of a synthesis option and about right for one pipeline cut — which is exactly what fixed it. Doing this arithmetic first saves you from trying the cheap fixes on a problem too big for them.

Change one thing per run, or you will not know which change helped.



Hold Does Not Care What the Clock Period Is

Hold does not care what the clock period is

The same design analysed at three wildly different frequencies.

clock period	frequency	hold slack	verdict
4.0 ns	250 MHz	-0.119 ns	VIOLATED
40.0 ns	25 MHz	-0.119 ns	VIOLATED
400.0 ns	2.5 MHz	-0.119 ns	VIOLATED

A hundred-fold change in frequency. No change at all in the violation.

Both flops are triggered by the SAME edge, so the period never enters the race. This is a functional failure, not a performance one - the chip is broken at every speed.

So the fix is the clock tree, or the data path - never the clock frequency

Rebalancing the tree so the capture register sees 0.10 ns of skew instead of 0.25 ns:

-0.119 ns VIOLATED -> +0.031 ns MET

A hundred-fold change in frequency. Not one picosecond of change in the violation.



Why It Is a Different Kind of Failure

SETUP violation

The data arrived too late for this clock.

Slow the clock down and the chip works.

You have shipped a less competitive product — but a working one.

Found at synthesis, and again after layout.

HOLD violation

The data changed too soon after the same edge.

There is no clock frequency at which the chip works.

Reaching silicon with one means a re-spin.

Found only once a real clock tree exists.

So the fix is never the clock

It is the clock TREE — rebalance it so the capture register sees less skew — or the data path: insert delay. Measured here: dropping skew from 0.25 ns to 0.10 ns took the slack from -0.119 ns to $+0.031$ ns.



Why Hold Violations Turn Up So Late

Why hold violations turn up so late in a project

RTL and synthesis

no clock tree exists at all, so skew is zero

hold passes trivially - and means nothing

Clock-tree synthesis

the tree is built; real skew appears

hold violations arrive, often in their hundreds

Place and route

buffers inserted on the offending data paths

hold closes; setup gets slightly worse

This is normal, and it is why hold is a layout problem more often than an RTL one

What RTL can do is not make it impossible: no hand-gated clocks, no unsynchronised crossings, no logic on an asynchronous reset.

What RTL can do about it: not make it impossible

No hand-gated clocks — use a clock enable. No unsynchronised domain crossings. No logic on an asynchronous reset. Beyond that, hold is place-and-route's job, and it has the one thing you do not: the real clock tree.



Practical Examples of Violations, and Their Solutions

Symptom	Likely cause	Solution
one path fails, the rest are fine	a genuinely long path	restructure or pipeline that path
one cell has a huge incremental delay	high fanout or a weak driver	set_max_fanout; let the tool buffer it
fifty small cells in one path	the logic is too deep	balance it into a tree, then pipeline
every path fails by a similar amount	the period is too aggressive	check the target against the technology
fails only after place-and-route	routing delay, not logic	floorplan; keep the path local
fails only at the slow corner	correct behaviour	that is the corner you sign off at
hold fails on a short path after CTS	clock skew	rebalance the tree; insert hold buffers
hold fails everywhere, at every speed	a structural problem	look for gated clocks and domain crossings
a huge violation on a reset path	a missing false path	constrain it properly — this is not a design bug



THEORY 3 · CHECKPOINT

Eight Questions

#	Question	The answer in one line
1	Write the setup inequality.	$t_{cq} + t_{logic} \leq T + \text{skew} - \text{setup} - \text{uncertainty}$
2	Which term is missing from the hold inequality?	the clock period T
3	Setup violation — what does the chip do?	works, at a lower frequency
4	Hold violation — what does it do?	does not work at any frequency
5	Late capture clock helps which check?	setup; it hurts hold by the same amount
6	Which stage sets Fmax?	the slowest one, on its own
7	−0.322 ns on a 2.5 ns period — how big a problem?	13%; pipeline territory, not a tool setting
8	Why do hold problems appear after CTS?	before it there is no clock tree, so no real skew

Theory 4 asks what synthesis does with everything you have just decided.

THEORY 4

Timing Constraints for Synthesis

A constraint file is not a report setting. It is the objective the optimiser is trying to meet — and it decides what circuit you get.

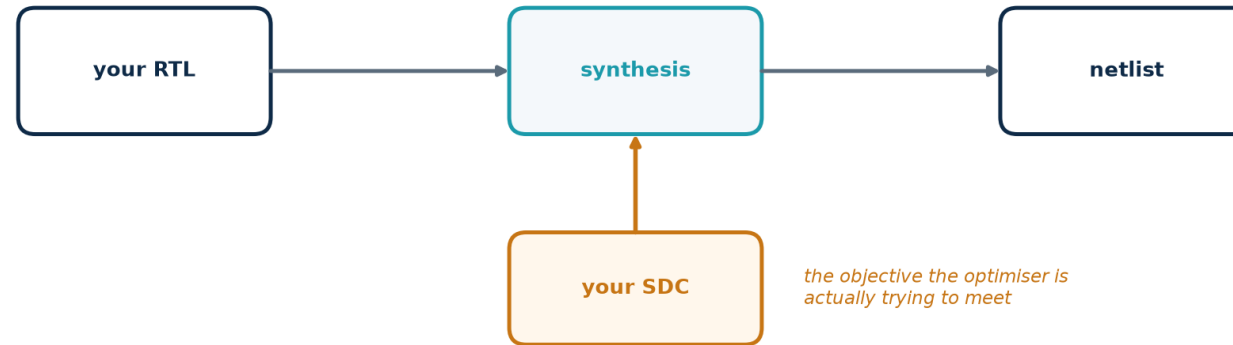
- What constraints actually do to synthesis
- The minimum honest constraint set
- What synthesis does to a hazard fix (it deletes it)
- Circuit synthesis and timing analysis as one loop



Constraints Are an Input to Synthesis

Constraints are the input to synthesis, not a report setting

The same RTL becomes a different circuit depending on what you asked for.



what you write	what the tool does	what you get
no constraint at all	optimise for area	small and slow
a loose period	stop as soon as it fits	small, just fast enough
a tight period	spend area on the critical path	large and fast
an impossible period	try everything, still fail	large, slow AND late

Row four is real, and it is expensive.

Ask for a period the technology cannot deliver and the tool burns area and runtime chasing it.

The same RTL becomes a different circuit depending on what you asked for.



The Minimum Honest Constraint Set, In Order

The minimum honest constraint set, in order

1

create_clock

Every clock, including generated ones. Without this there is no target and nothing is checked.

2

set_clock_uncertainty

Jitter, plus the clock-tree skew that does not exist yet. Taking it out makes the report prettier and the chip no faster.

3

set_input_delay / set_output_delay

Every port, against the clock that samples it. Omit these and every boundary path is UNCONSTRAINED - unchecked, not passed.

4

exceptions, each with a reason

set_false_path and set_multicycle_path. Only write one you can justify in a sentence on the line above it.

5

check for holes

Report unconstrained endpoints. It must be zero. A clean report on a half-constrained design is worse than a failing one.

Module 2 Topic 6 develops each of these in full.

This topic assumes them and moves on to what synthesis then does with your circuit.

Module 2 Topic 6 develops each of these in full. Here the point is what synthesis does once you have written them.



A Complete File For the Lab Design

```
# ===== constraints/pipe.sdc
# ---- clock -----
# 400 MHz. pipe_unbal cannot make this; pipe_bal can. That is the experiment.
create_clock -period 2.500

# jitter, plus a placeholder for the clock-tree skew that does not exist yet
set_clock_uncertainty 0.080 -setup
set_clock_uncertainty 0.020 -hold

# ---- boundary -----
# Without these two lines every path touching a port is UNCONSTRAINED,
# and the analyser says so. Run without them once and read the warning.
set_input_delay 0.40 -clock clk
set_output_delay 0.35 -clock clk
```

The line to check on every single run

The count of unconstrained endpoints. It must be zero. A design with WNS +2.0 ns and four hundred unconstrained endpoints is in far worse shape than one with WNS -0.1 ns and none — the first has not been analysed at all.



Synthesis Deletes Your Hazard Fix. Every Time.

Synthesis deletes your hazard fix. Every time.

The consensus term is REDUNDANT by construction. Removing redundancy is what an optimiser is for.

what you wrote in RTL	cells	gate	result
$f = a\&\sim b \mid b\&c$	1	<code>\$_MUX_</code>	the hazard is still in there
$f = a\&\sim b \mid b\&c \mid a\&c$	1	<code>\$_MUX_</code>	identical netlist - term gone

And look at what it built instead

A single multiplexer. $A B' + B C$ is exactly $B ? C : A$, and the optimiser saw that before it saw anything else. Whether THAT cell glitches is decided by its internals, in a library you did not write.

So how do you keep a hazard-free circuit?

- Protect it structurally: a `dont_touch` / `keep` attribute on the net or the cell.
- Or instantiate a library cell directly, so there is nothing to optimise.
- Or put it in a module the tool is told not to flatten.
- And then re-run the glitch detector on the NETLIST, not on the RTL.

Both RTL versions produced the identical netlist: one multiplexer.



Why This Is the Most Important Slide In the Session

Theory 2 taught you to add a redundant term to remove a hazard. Theory 4 has just shown that a synthesiser deletes it. Those two facts have to be reconciled, and the reconciliation is the practical skill.

What NOT to conclude

"Hazard analysis is pointless because the tool undoes it."

"Just write the RTL and trust synthesis."

Both are wrong, and the second one ships broken asynchronous interfaces.

What to conclude

Hazard-freedom is a property of the NETLIST, not of your source.

So it has to be protected structurally, and then verified on the netlist.

And on the vast majority of nets — everything sampled by a clock — you do not need it at all.

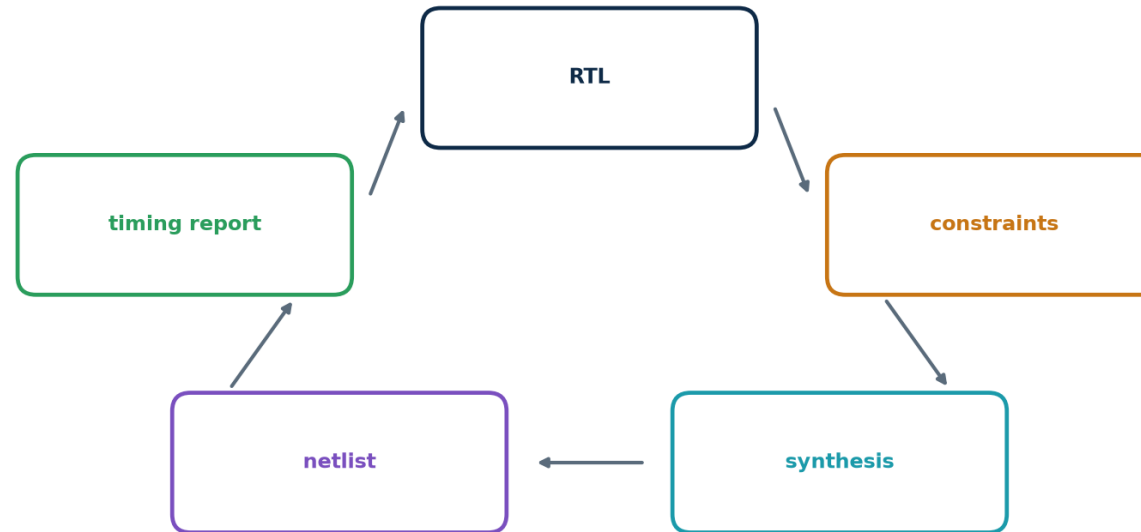
How to protect it, when you genuinely need it

A dont_touch or keep attribute on the net or cell · instantiate the library cell directly · put it in a module the tool is told not to flatten · and then re-run the glitch detector on the post-synthesis netlist, not on the RTL.



Circuit Synthesis and Timing Analysis

Circuit synthesis and timing analysis: one loop



You go round this loop until three things are true at once

- ✓ setup slack ≥ 0 at the slow corner
- ✓ hold slack ≥ 0 at the fast corner
- ✓ zero unconstrained endpoints

You leave the loop once, when all three conditions hold together. Two out of three is not closure.



Everything In This Topic Is a Trade

Everything in this topic is a trade

you pay

you get

hazard-free cover

+ area, + power

no glitches on that output

pipelining

+ registers, + latency

a faster clock

hold buffers

+ area, + power

the design works at all

tighter constraints

+ area, + runtime

a faster chip, up to a point

looser constraints

- area

a slower chip you can actually build

There is no free row in that table, and a design that pays nothing gets nothing.

The engineering is in deciding which of these you need, on which nets, and defending the choice.

There is no free row in that table, and a design that pays nothing gets nothing.



Six Questions

#	Question	The answer in one line
1	What does a constraint file do to synthesis?	it is the objective the optimiser tries to meet
2	What happens with no constraints at all?	the tool optimises for area; you get small and slow
3	What happens with an impossible period?	it burns area and runtime and still fails
4	What did synthesis do to the consensus term?	deleted it — both versions gave one identical MUX
5	So where does hazard-freedom have to be checked?	on the netlist, never on the RTL
6	When do you leave the synthesis/analysis loop?	setup met, hold met, zero unconstrained endpoints

That is the theory. The rest of the session is at the keyboard.

PRACTICAL

Labs A-G

12 hours, 58 graded exercises, and not one number in this deck that you cannot reproduce yourself.

- Which tool answers which question, and how to install them
- Labs A-C: hazards, on paper, in code, and in simulation
- Labs D-E: does it matter, and what synthesis does about it
- Labs F-G: setup, hold, Fmax — then the same design on a Zynq-7000



Which Tool Answers Which Question

Which tool answers which question

tool	licence	the question it answers	used in
tools/hazard.py	you write it	which covers have hazards	this lab
Icarus Verilog	free	does it actually glitch, and when	this lab
GTKWave	free	look at the glitch	this lab
Yosys	free	what does synthesis build	this lab
sta/sta.py	you write it	setup, hold, slack, Fmax	Module 2 T6
Vivado	free WebPACK	all of the above, on a real device	syllabus tool
ModelSim / Questa	free starter	gate-level simulation with SDF	syllabus tool
PrimeTime / Tempus	commercial	ASIC sign-off timing	industry

The pairing that matters in this topic

A static analyser for setup and hold, and a delay-annotated simulator for hazards. Neither one substitutes for the other, and a flow with only one of them has a blind spot.

Everything except the last three rows runs on free software you can install in one command.

A static analyser for setup and hold, a delay-annotated simulator for hazards. Neither substitutes for the other.



One apt Line, and the Whole Lab Runs

Install this, and the whole lab runs

One apt line. No licence, no account, no download manager.

REQUIRED

Debian / Ubuntu / WSL2

```
sudo apt update
```

```
sudo apt install yosys iverilog gtkwave python3 python3-matplotlib
```

Yosys synthesis - and what it does to your hazard fix

Icarus gate-level simulation WITH DELAYS - this is the one

that shows you a glitch; nothing else in the flow can

GTKWave to look at the VCD when the counter says something odd

Python 3 runs tools/hazard.py and sta/sta.py

```
yosys -V && iverilog -V && python3 tools/hazard.py --selftest
```

*The self-test cross-checks the hazard rule against a timing simulation on 400 random functions.
If it prints PASSED, everything in this topic will run.*

Debian / Ubuntu / WSL2 — copy this

```
sudo apt update
sudo apt install yosys iverilog gtkwave python3 python3-matplotlib
yosys -V && iverilog -V && python3 tools/hazard.py --selftest
```

On Windows: wsl --install first, then the same three lines inside it.



The Vendor Tools the Syllabus Names

The vendor tools the syllabus names

Needed for the last lab only. Everything before it runs without them.

	what to do	watch out for
1	Create a free AMD/Xilinx account	the download will not start without one
2	Download the Unified Installer (web installer, ~200 MB)	not the 40 GB full archive
3	Choose Vivado ML Edition, then Vivado ML Standard	Standard is free; no licence file
4	Deselect every device family except Zynq-7000	cuts 40 GB to about 8 GB
5	Verify with <code>vivado -version</code>	then it runs headless
6	The ZynQ7000 board and JTAG cable are optional here	this is a synthesise-and-report flow

You do not need the board to do the timing work

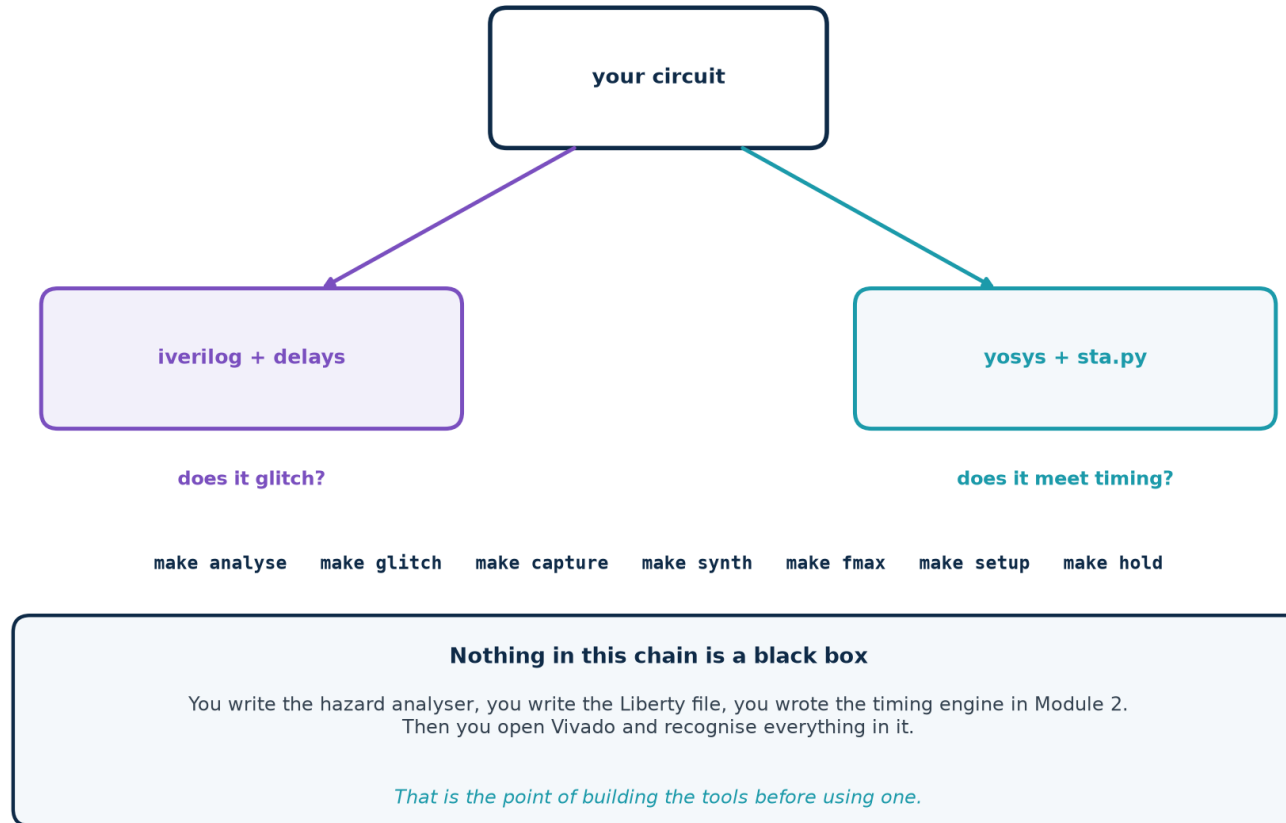
Vivado will target a Zynq-7000 it has never been connected to and give you a real timing report for it. The board and the JTAG cable matter when you want to run the design, not analyse it.

Needed for lab G only. Labs A-F — 53 of the 58 exercises — run on the free toolchain alone.



One Circuit, Two Toolchains

The lab flow: one circuit, two toolchains



You write the hazard analyser here; you wrote the timing engine in Module 2. Nothing in the chain is a black box.



Seven Parts, Twelve Hours

The practical component - 12 hours, seven parts

Module 3 practical: 35 hours across three subtopics. This is subtopic 1's share.

A	Hazards on paper	K-maps, adjacency, the consensus term	2 h
B	Build the analyser	tools/hazard.py, and its self-test	2 h
C	Gate-level glitches	the detector; static, then dynamic	2 h
D	Does it matter?	one signal as data, as clock, as reset	1 h
E	Synthesis	watch the optimiser delete your fix	1 h
F	Setup, hold, Fmax	the pipeline, the skew, the period	2 h
G	Vivado on Zynq-7000	the same design in an industrial tool	2 h
58 graded exercises, every one with a worked solution Parts A-C build the understanding and the tool. Parts D-E connect it to real design decisions. Parts F-G are the timing half, and part G proves the ideas transfer. <i>Nothing is quoted in this topic that you cannot reproduce with make.</i>			

Parts A-C build the understanding and the tool. D-E connect it to design decisions. F-G are the timing half.



Hazards: On Paper, In Code, In Simulation

A · on paper (2 h)

Draw the K-map for $F = A B' + B C$.

Find the adjacent 1s that share no loop.

Derive the consensus term by hand.

Deliverable: the term, and a sentence saying why it works.

B · in code (2 h)

Write the adjacency test in `tools/hazard.py`.

Make the self-test pass, including the cross-check against a delay timeline.

Deliverable: PASSED, and an explanation of what section D of it proves.

C · in simulation (2 h)

Build the glitch detector.

Predict the glitching transition BEFORE running it.

Then the dynamic case, and its two different fixes.

Deliverable: the six-row results table, explained.

The exercise that teaches the most is C

Predict which of the 24 transitions will glitch, from the analyser's output, before you simulate. When the prediction matches, you have connected a piece of logic reasoning to a physical measurement — which is the whole of this subtopic.



Does It Matter, and What Synthesis Does About It

both results, measured

```
$ make capture
f as DATA, sampled by the clean clock  -> correct
f as a CLOCK          edges after power-up = 4  <- should be 0
f as an ASYNC RESET  r_flag = 0                <- should be 1

$ make synth
f = a&~b | b&c          1 cell  {'$_MUX_': 1}
f = a&~b | b&c | a&c  (fixed) 1 cell  {'$_MUX_': 1}
The two netlists are IDENTICAL. The consensus term was deleted.
```

Exercise	Task
D.1	Move the glitch to 5 ns before the clock edge. Does the DATA column still hold?
D.2	Replace the async reset with a synchronous one. Re-measure.
D.3	Name three signals in a design you have written that are edge-sensitive.
E.1	Add a keep attribute to the consensus term. Does it survive?
E.2	Re-run the glitch detector on the post-synthesis netlist.
E.3	Explain in writing why the tool chose a MUX.



Setup, Hold, Fmax — Then an Industrial Tool

```
$ make fmax
pipe_unbal    Fmax : 364.7 MHz
pipe_bal      Fmax : 473.2 MHz

$ make setup          # target 400 MHz
pipe_unbal    WNS : -0.322 ns  VIOLATED
pipe_bal      WNS : +0.307 ns  MET

$ make hold
period    4.0 ns    WNS : -0.119 ns  VIOLATED
period   40.0 ns    WNS : -0.119 ns  VIOLATED
period  400.0 ns    WNS : -0.119 ns  VIOLATED
skew 0.25 -> 0.10  WNS : -0.119 -> +0.031 ns  MET
```

Lab G: the same design on a Zynq-7000

vivado -mode batch -source vivado/zynq_sta.tcl, targeting xc7z020clg400-1. The absolute numbers will differ — a Zynq LUT is not the teaching library. What must match is the shape of the report, which path is critical, and the direction of every constraint change.



How the 58 Exercises Are Weighted

How the 58 exercises are weighted

part	exercises	weight	assessed on
A · hazards on paper	10	15%	correct adjacency reasoning
B · the analyser	8	15%	self-test passes, and you can explain it
C · gate-level glitches	12	20%	prediction before measurement
D · does it matter	6	10%	the right rule, not the slogan
E · synthesis	6	10%	what survived, and why
F · setup, hold, Fmax	11	20%	hand arithmetic matches the tool
G · industrial tools	5	10%	the comparison write-up

The rule that runs through every part

A fix you cannot explain is not a fix. Marks go to the reasoning, not to a clean report - because in this field it is trivially easy to produce a clean report on a broken design.

A fix you cannot explain is not a fix.



What This Lab Does Not Show You

- The gate delays are teaching-sized. A real inverter is not 4 ns; the mechanism is real, the timescale is not.
- The delay numbers in the Liberty file are invented, not any vendor's. They are chosen to be checkable by hand.
- sta/sta.py is a teaching analyser: straight-line delays, no slew propagation, no wire RC, one corner at a time.
- The hazard analyser handles two-level covers. Multi-level hazard analysis is a much harder problem, and hz_dynamic shows you why.
- vivado/zynq_sta.tcl has NOT been executed in the environment these materials were built in — Vivado is not installable there. It is the one file whose output is not reproduced in the README.
- Nothing here covers on-chip variation, crosstalk, or signal integrity. Those belong to sign-off, in subtopic 3.



Topic 1 In Ten Lines

- A truth table promises settled values. It says nothing about the journey.
- A hazard is the circuit's potential to glitch; a glitch is one event in one run.
- Two 1-cells one variable apart, covered by no single term, is a static-1 hazard.
- The fix is the term made of the literals they agree on — redundant, by design.
- That fix cures two-level logic hazards and nothing else. Structure needs restructuring.
- A glitch is harmless only where a clock edge samples it after it settles.
- Static timing analysis cannot see any of this: it measures one path, not two.
- The clock period is in the setup equation and nowhere in the hold equation.
- Fmax is set by one stage; moving work took 364.7 MHz to 473.2 MHz.
- And synthesis will delete your hazard fix, so verify on the netlist.



CLOSE

What To Do Next

By the end of this subtopic you can

- ✓ Look at a two-level cover and say whether it has a static hazard, and which transition.
- ✓ Prescribe the redundant term that removes it, and prove the function did not change.
- ✓ Build a testbench that detects glitches by counting, not by looking.
- ✓ Say where a glitch is harmless and where it is fatal - and defend the distinction.
- ✓ Explain why static timing analysis cannot see any of this.
- ✓ Compute maximum frequency of operation from the four terms of a path.
- ✓ Diagnose a setup violation, size it against the period, and pick the right fix.
- ✓ Recognise a hold violation and explain why the clock frequency is irrelevant to it.
- ✓ Write a constraint set that makes synthesis optimise for what you actually want.

And one thing to carry into every design you ever do

"It simulates correctly" and "it meets timing" are two different claims, and neither implies the other.

Coming next in Module 3

Subtopic 2 takes the optimisation techniques further — pipelining, retiming, open- and closed-loop timing, block-level versus chip-level. Subtopic 3 is the EDA flow end to end: post-route analysis, ECO, and sign-off.